# Efficient Learning-based Top-$k$ Representative Similar Subtrajectory Query

Kunming Wang[†], Shiyu Yang[†*], Jiabao Jin[§], Peng Cheng[§], Jianye Yang[†], Xuemin Lin[‡]
[†]*Guangzhou University;* [§]*East China Normal University;* [‡]*Shanghai Jiao Tong University*
wkm82500@outlook.com; syyang@gzhu.edu.cn; jiabaojin@stu.ecnu.edu.cn
pcheng@sei.ecnu.edu.cn; jyyang@gzhu.edu.cn; xuemin.lin@sjtu.edu.cn

***Abstract*—The advancement in location technology and the increase in trajectory data have made trajectory data mining a focal point in the field of spatio-temporal data analysis. Unlike the extensive research focused on the similarity of entire trajectories, this paper delves into subtrajectory similarity within collections containing a large number of trajectories, under a specified trajectory similarity metric. We propose the Top-$k$ Representative Similar Subtrajectory Query problem, with the objective of identifying the top-$k$ representative subtrajectories which are most similar to the query trajectory within a large trajectory set. To ensure diversity in the results, we adopt the concept of representative similarity, where only the subtrajectory with the highest similarity score to the query trajectory is reported, thus avoiding the redundancy in the top-$k$ results. To address the challenge of high computational costs, we propose a learning-based framework, leveraging a deep learning model called Representative Similarity Score Estimation (RSSE) to approximate subtrajectory similarity scores efficiently and reduce the candidate set significantly. Empirical evaluations conducted on various real-world datasets substantiate the effectiveness and efficiency of our proposed method.**

## I. Introduction

In recent years, spatial-temporal data mining, including trajectory mining, has been growing field of study attracting significant research interest, driven by the increasing ubiquity of location-based services and the proliferation of devices capable of capturing spatial and temporal data. Trajectory mining has found myriad applications, ranging from traffic flow prediction [1] [2] and travel time estimation [3] [4] to intricate analyses such as athlete behavior pattern studies [5]. Trajectory similarity is a fundamental concept in many of these applications. Specifically, the task of identifying trajectories similar to a given query trajectory, according to trajectory similarity metrics such as Dynamic Time Warping (DTW) [6], Edit Distance on Real sequence (EDR) [7], Edit distance with Real Penalty (ERP) [8], and others, has become fundamental and is referred to as the trajectory similarity search problem.

The similarity search of the entire trajectory [9]–[14] has been extensively studied. However, it may overlook some important candidates due to the difference in length. As illustrated in Figure 1, under the trajectory similarity measure, the overall similarity between the query trajectory $T^q$ and the data trajectories ($T^{d1}$ and $T^{d2}$) is low. This can be attributed to the considerable length of the data trajectories, resulting in certain portions of these trajectories being significantly distant from

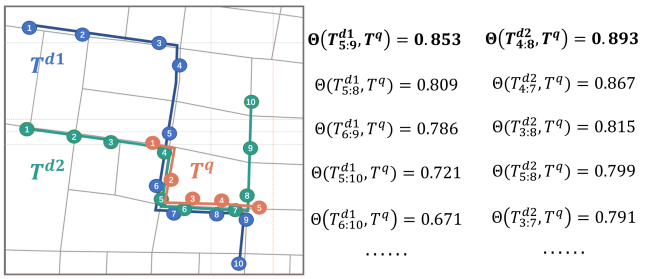



Fig. 1: An example of representative similar subtrajectory, For $T^q$, the representative similar subtrajectory in $T^{d1}$ is $T^{d1}_{5:9}$, and in $T^{d2}$, it is $T^{d2}_{4:8}$, with corresponding representative similarity scores of 0.835 and 0.893, respectively.

$T^q$. As a result, $T^{d1}$ and $T^{d2}$ will not be considered during the similar trajectory search process. It is noteworthy that certain continuous trajectory segments referred to as subtrajectories in $T^{d1}$ and $T^{d2}$ that cover $T^q$ (i.e., $T^{d1}_{5:9}$ and $T^{d2}_{4:8}$) and display significant similarity to $T^q$.

To capture the trajectory similarity in a fine-grained mode as shown in the above example, substrajectory analysis is a promising option. One motivating application is the travel time estimation for a given path. Recent methods [15] rely on retrieving historical trajectories from a trajectory database that are similar to the query path. However, in practice, the number of trajectories with the same starting and ending points as the query path is limited, which may result in significant estimation bias. At the mean while, many historical trajectories pass through the query path, and some of their subtrajectories are similar to the query path, potentially providing more meaningful results. Another interesting application is the detour detection for a given route. If the current route is similar to a known detour route, the current route may also be a detour. By considering subtrajectory similarity instead of treating the trajectory as a whole, we can potentially detect detours earlier, as the current route may exhibit similarity to a subtrajectory of the detour route.

Subtrajectory offers a more fine-grained approach to analyzing trajectory similarity. However, retrieving similar subtrajectories from a trajectory database can introduce redundancy. As illustrated in Figure 1, subtrajectories $T^{d2}_{4:8}$ and $T^{d2}_{4:7}$ of $T^{d2}$ exhibit similarity scores of 0.893 and 0.876 with $T^q$, respectively, surpassing the score of subtrajectory $T^{d1}_{5:9}$ of $T^{d1}$

*Shiyu Yang is the corresponding author

2024 IEEE 40th International Conference on Data Engineering (ICDE) | 979-8-3503-1715-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICDE60146.2024.00335



2375-026X/24/$31.00 ©2024 IEEE
DOI 10.1109/ICDE60146.2024.00335

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

with $T^q$. However, the marginal difference between $T^{d2}_{4:8}$ and $T^{d2}_{4:7}$, attributed to the absence of a single point, renders them redundant. Essentially, subtrajectories within a trajectory that marginally deviate from the most similar subtrajectory to $T^q$ often maintain high similarity with $T^q$, yet fundamentally convey much the same information. To address this issue, we propose to retain only the subtrajectory of each trajectory with maximal similarity score to $T^q$, termed as the representative similar subtrajectory and establish representative similar subtrajectories as the primary unit for similarity research. Furthermore, given the vastness of trajectory databases, housing millions of trajectories, retrieving an exhaustive list of similar subtrajectories can be overwhelming. Utilizing a top-$k$ approach enhances the efficiency of results by concentrating on the most relevant results. In many applications, such as travel time estimation, a top-$k$ result set suffices to meet user requirements and ensures optimal performance. Based on the above observation, we propose the **T**op-$k$ **R**epresentative **S**imilar **S**ubtrajectory **Q**uery (TRSSQ) problem, which seeks to retrieve the top-$k$ representative similar subtrajectories from a trajectory database.

**Challenges.** A naive approach to solve the TRSSQ problem involves two main steps. First, for each trajectory in the database, the method enumerates all possible subtrajectories and calculates their similarity scores with respect to the query trajectory. The subtrajectory with the highest similarity score is then designated as the representative similar subtrajectory for the query trajectory. This process is repeated for every trajectory in the database. Once the representative similar subtrajectories have been determined for all trajectories in the database, the top-$k$ representative similar subtrajectories with the highest similarity scores are returned as the result.

However, this naive approach suffers from several drawbacks. Identifying the representative similar subtrajectory within a single trajectory requires evaluating all possible subtrajectories, which can be computationally expensive. The exact algorithm Exacts [16] for computing the subtrajectory with maximal similarity entails a high computational cost, making it impractical for large-scale trajectory databases. Although heuristic algorithms [16] offer lower computational complexity, they often fail to accurately identify the most similar subtrajectories, leading to suboptimal top-$k$ results. Moreover, existing filtering methods [17] for similar subtrajectories are ineffective at eliminating trajectories that do not contain similar subtrajectories, posing a significant challenge in efficiently addressing the TRSSQ problem within large trajectory databases.

**Motivation and Solution.** Identifying representative similar subtrajectories within a data trajectory provides two pieces of information: their position within the data trajectory and their representative similarity scores, which determine whether a subtrajectory belongs to the top-$k$ results. Top-$k$ representative similar subtrajectories refer to the $k$ representative similar subtrajectories with the highest representative similarity scores among all candidate trajectories. Based on these observations, we propose to separate the process of identifying representative similar subtrajectories from locating them within the data trajectory and obtain their representative similarity scores.

We propose a Representative Similarity Score Estimation (RSSE) model to rapidly estimate the representative similarity score between a given query trajectory and data trajectories. The RSSE model is able to estimate the highest similarity score among all possible subtrajectories of a data trajectory to the query trajectory without computing the similarity score of each individual subtrajectory of the data trajectory. This allows us to select trajectories with high estimated similarity scores as candidates, effectively excluding a large portion of trajectories unlikely to contain the top-$k$ results and eliminating the time-consuming similarity score computation for unnecessary trajectories. Consequently, the efficiency of TRSSQ processing is significantly enhanced. We also claim that the RSSE model is applicable to various trajectory similarity metrics.

Specifically, as illustrated in Figure 2, we adopt a filtering and verification paradigm for the TRSSQ problem, employing RSSE to predict the representative similarity scores between a query trajectory $T^q$ and all trajectories in the database. The accuracy of RSSE directly influences candidate quality and, ultimately, TRSSQ result quality. A user-defined parameter $M$, larger than $k$ but much smaller than the total number of trajectories in the database, is introduced to determine the size of the candidate set. A larger number of candidates enables RSSE to achieve higher candidate quality. In the verification phase, we identify the representative similar subtrajectories within each candidate trajectory by computing their representative similarity scores to ultimately derive the top-$k$ results.

To summarize, we make the following contributions in this paper:

- We formulate the TRSSQ problem, which aims to find the $k$ most similar subtrajectories to a query trajectory while ensuring diversity in the results. To the best of our knowledge, we are the first to propose this problem.
- We propose a learning-based framework designed to efficiently address the TRSSQ problem. This framework leverages our meticulously crafted Representative Similarity Score Estimation model(RSSE) to enable efficient similarity score estimation.
- We conducte extensive experiments on real-world datasets to validate the effectiveness and efficiency of our proposed method.

## II. Preliminaries

In this section, we begin by defining key terms and concepts, then formally propose the TRSSQ problem. Finally, we describe the trajectory similarity metrics employed in this study.

### A. Problem Definition

**Definition 1** (Trajectory)**.** A trajectory with length $n$ is a sequence of locations at different timestamps which can be represented as $T = (p_1, p_2, \ldots, p_n)$, where each position $p_i$ usually consists of longitude, latitude coordinates and timestamp $t_i$ . For example, $p_i = (t_i, lon_i, lat_i)$ indicates the location at $lon_i, lat_i$ at time $t_i$. For ease of representation,



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

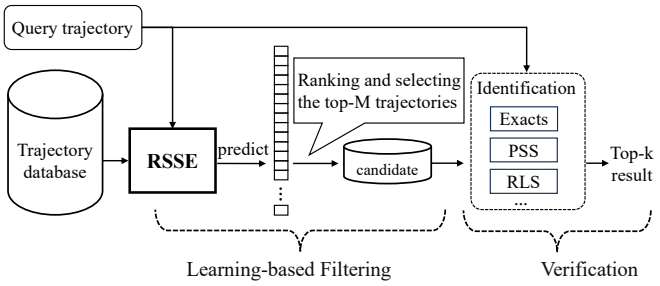



Fig. 2: Framework of learning-based top-$k$ representative similar subtrajectory query

we usually omit the time dimension $t_i$. A trajectory database contains $n$ trajectories, notated as $D$=$\{T_1, T_2, \ldots, T_n\}$.

**Definition 2** (Subtrajectory)**.** Given a trajectory $T = (p_1, p_2, \ldots, p_n)$, a subtrajectory denoted as $T_{i:j}$ of $T$ is a continuous segment in $T$ with $p_i$ as the start point and $p_j$ as the end point, where $1 \leq i < j \leq n$.

**Definition 3** (Representative Similar Subtrajectory)**.** For a given data trajectory $T_i^d$ with length $m$ in database $D$, a query trajectory $T^q$ with length $n$, typically, m>n. The representative similar subtrajectory $R_i$ is the subtrajectory of $T_i^d$ that maximizes the similarity with $T^q$ under a predefined trajectory similarity metric $\Theta(\cdot,\cdot)$. Formally, $R_i$ is defined as: $R_i = T^d_{a:b}$, where $(a,b) = \underset{1\leq a\leq b\leq m, T^d_{a:b}\in T_i^d}{\arg\max} \Theta(T^q, T^d_{a:b})$ and $\Theta(T^q, R_i)$ is the representative similarity score.

**Definition 4** (Top-$k$ Representative Similar Subtrajectory Query)**.** Given a query trajectory $T^q$, a trajectory database $D$ contains $n$ trajectories, and a trajectory similarity metric $\Theta(\cdot,\cdot)$, let $R = \{R_1, R_2, \ldots, R_n\}$ be the set of representative similar subtrajectories obtained from each data trajectory in $D$, TRSSQ problem aim to obtain a subset $R' \in R$ as follow:

$$R' = \underset{R'\subseteq R, |R'|=k}{\operatorname{argmax}} \sum_{x\in R'} \Theta(x, T^q) \tag{1}$$

### *B. Trajectory Similarity Metrics*

As mentioned in Section I, various metrics are employed to gauge the similarity of trajectories. In this study, we select four widely recognized similarity metrics from the literature: DTW, EDR, ERP, and LCSS. We denote the similarity functions as $\Theta^{DTW}, \Theta^{ERP}, \Theta^{EDR}$, and $\Theta^{LCSS}$, respectively. We herein define each metric, where $d(\cdot,\cdot)$ signifies the distance function between two points.

**DTW.** Given two trajectories $T^q$ with length $i$ and $T^d$ with length $j$, the DTW distance can be defined as follows:

$$\Theta^{DTW}_{i,j} = \begin{cases} \sum_{h=1}^{\max(i,j)} d(T^q_{\min(h,i)}, T^d_{\min(h,j)}), & i=1 \text{ or } j=1 \\ d(T_i^q, T_j^d) + \\ \min(\Theta^{DTW}_{i-1,j}, \Theta^{DTW}_{i,j-1}, \Theta^{DTW}_{i-1,j-1}), & \text{otherwise} \end{cases} \tag{2}$$

Here, we have $\Theta^{DTW}_{i,j} = \Theta^{DTW}(T^q_{1:i}, T^d_{1:j})$. DTW allows multiple points in the query trajectory $T^q$ to be mapped to a point in $T^d$. It is clear that the time complexity of computing the DTW distance according to Equation 2 is $O(nm)$.

**ERP.** In contrast to DTW, ERP allows only one point to be replaced by another and removes mismatched points by insertion and deletion. By setting a default point $p'$ (i.e., often in the center of the map) in advance, ERP records the cost of deletion, insertion and replacement as $del(T_i^q) = d(T_i^q, p')$, $ins(T_j^d) = d(T_j^d, p')$ and $sub(T_i^q, T_j^d) = d(T_i^q, T_j^d)$. Therefore, the calculation process of the ERP distance can be expressed as follows:

$$\Theta^{ERP}(T^q_{1:i}, \varepsilon) = \sum_{h=1}^{i} del(p_q^{(h)})$$

$$\Theta^{ERP}(\varepsilon, T^d_{1:j}) = \sum_{k=1}^{j} ins(p_t^{(k)})$$

$$\Theta^{ERP}_{i,j} = \min \begin{cases} del(T_i^q) + \Theta^{ERP}_{i-1,j} \\ ins(T_j^d) + \Theta^{ERP}_{i,j-1} \\ sub(T_i^q, T_j^d) + \Theta^{ERP}_{i-1,j-1} \end{cases} \tag{3}$$

**EDR.** Unlike ERP, EDR sets the cost of insertion, deletion, and replacement operations by employing a threshold rather than using a default value for $p'$. It defines the cost of both insertion and deletion as 1, that is $del(T_i^q) = 1, ins(T_j^d) = 1$. The cost of substitution $sub(T_i^q, T_j^d)$ is contingent upon $d(T_i^q, T_j^d)$. Given a threshold $\gamma$, if $d(T_i^q, T_j^d) > \gamma$, then $sub(T_i^q, T_j^d) = 1$; otherwise, it is 0.

**LCSS.** Similar to EDR, the LCSS similarity metric employs a parameter $\epsilon$ to determine whether two points match,where similarity is denoted by the length of the longest common subsequence. LCSS similarity metric is defined in Equation 4.

$$\Theta^{sLCSS}_{i,j} = \begin{cases} 0, & i=0 \text{ or } j=0 \\ 1 + \Theta^{sLCSS}_{i-1,j-1}, & d(T_i^q, T_j^d) \leq \epsilon \\ \max(\Theta^{sLCSS}_{i-1,j}, \Theta^{sLCSS}_{i,j-1}), & \text{otherwise} \end{cases} \tag{4}$$

$$\Theta^{LCSS}_{i,j} = i + j - 2 \times \Theta^{sLCSS}_{i,j} \tag{5}$$

The number of irrelevant points in trajectories also affects the similarity of trajectories. Hence, we use LCSS distance [18] defined in Equation 5, which balances the shared and different parts between trajectories, making it well-suited for the TRSSQ problem.

## III. Representative Similarity Score Estimation Network

In this section, we present the RSSE network, designed to estimate the representative similarity scores between a given pair of query and data trajectory. Subsequently, we delve into the details of the two key modules of RSSE: the point-level matching module and the similarity extraction module. Finally, we explore the optimization objectives and the employed sampling method.



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

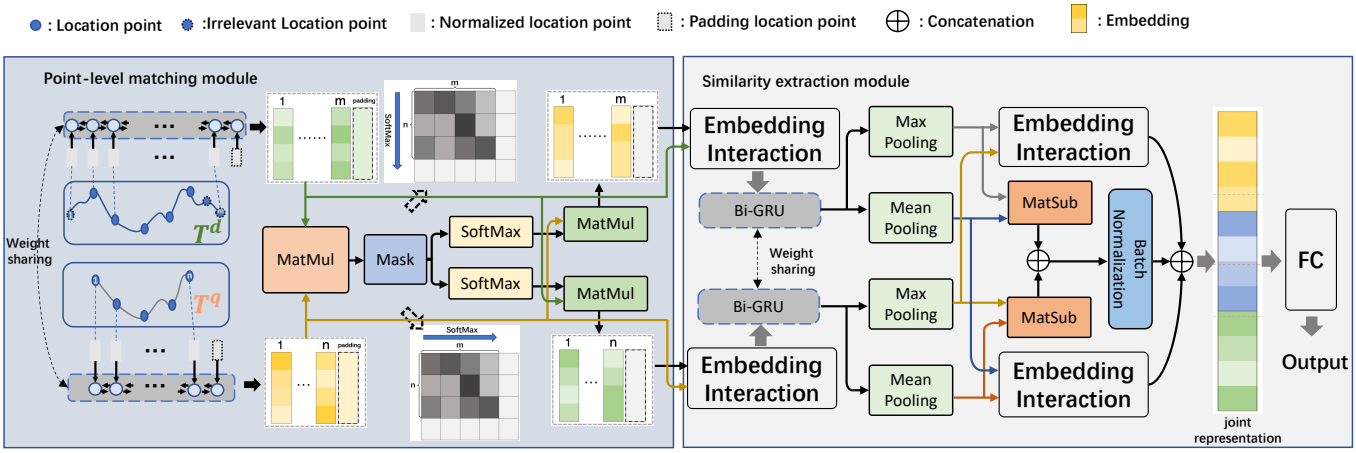



Fig. 3: Structure of RSSE model

### A. Overview

Existing research on learning-based trajectory similarity estimation has primarily focused on the similarity of entire trajectories. These methods use sequence models to encode entire trajectory information and capture sequence and positional details. However, these methods fail to accurately estimate the representative similarity score, which is solely related to the representative similar subtrajectories within the data trajectory. Encoding the entire trajectory generates embeddings that inevitably include information from points outside the representative similar subtrajectories, introducing biases in the prediction of representative similarity scores.

As illustrated in Figure 4, our analysis of point-level nearest matching in $T^q$ and $T^d$ reveals that points in $T^d$ belonging to representative similar subtrajectories are relatively closer to their nearest points in $T^q$, while irrelevant points have a relatively farther distance to their nearest point in $T^q$. We illustrate the distance between each point in a trajectory and its nearest counterpart in another trajectory using heat maps. Due to the continuity of subtrajectories, this distance-related characteristic is reflected as continuous red regions in heat maps. Based on the above analysis, we design the RSSE network, shown in Figure 3, to eliminate the influence of irrelevant points and focus on capturing the similarity of representative similar subtrajectories. The RSSE network consists of two main modules: the point-level matching module and the similarity extraction module. The point-level matching module employs Bi-GRU to encode the trajectory into embeddings and utilizes an attention mechanism to compute the embedding representation of the nearest point in one trajectory for points in the other trajectory. In the similarity extraction module, the distance between embeddings is computed through fine-grained similarity feature extraction to generate a joint representation, ultimately obtaining the representative similarity score.

### B. Point-level Matching Module

Given a query trajectory $T^q = (p_1^q, p_2^q, \ldots, p_n^q) \in \mathbb{R}^{n\times 2}$ and a data trajectory $T^d = (p_1^d, p_2^d, \ldots, p_m^d) \in \mathbb{R}^{m\times 2}$, where $n$ and $m$ denote the lengths of $T^q$ and $T^d$, respectively, RSSE employs the normalized location information as input features. In the context of mini-batches, each batch comprises multiple pairs, necessitating padding the query and data trajectories with zeros to match the length of the longest query and data trajectories, respectively. For brevity, we momentarily disregard padding in our discussion.

Trajectories, as temporally ordered sequences of locations, are encoded using recurrent neural networks (RNNs) to capture their sequential nature. The Gated Recurrent Unit (GRU), a simplified RNN variant with gating mechanisms for maintaining long-term dependencies, is utilized for its efficiency in embedding locations with sequential context. The Bidirectional GRU (Bi-GRU), enhancing GRU by integrating forward and backward sequence information, generates comprehensive trajectory embeddings. Each Bi-GRU time step's hidden layer embeds a location point, resulting in the trajectory's overall embedding represented as follows:

$$G^q = \text{Bi-GRU}(T^q), G^d = \text{Bi-GRU}(T^d) \tag{6}$$

Here, $G^q \in \mathbb{R}^{n\times d}$ and $G^d \in \mathbb{R}^{m\times d}$, with Bi-GRU transforming each geographic point into a $d$-dimensional embedding that incorporates sequence information.

As analyzed in Section III-A, the point-level matching plays a pivotal role in estimating representative similarity scores. We utilize an attention mechanism for this purpose. Taking point $G_i^d$ in $G^d$ as an example, the matching process is as follows:

$$M_i^d = \sum_{j=1}^{n} softmax(G_i^d * G_j^q) \cdot G_j^q \tag{7}$$

Here, the inner product operation $(*)$ meatures the match or distance between $G_i^d$ and $G_j^q$. To favor closer points while reducing the influence of distant points, softmax weights are applied to these distances. This approach assigns lower weights to more distant points in the summation, thereby preserving the information of the nearest neighbor points. $M_i^d \in \mathbb{R}^d$ represents the embedding of the matching point in



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

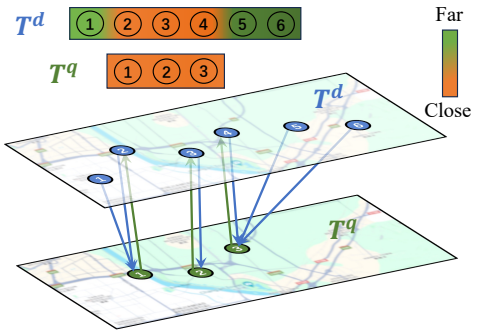



Fig. 4: Nearest point matching in $T^q$ and $T^d$

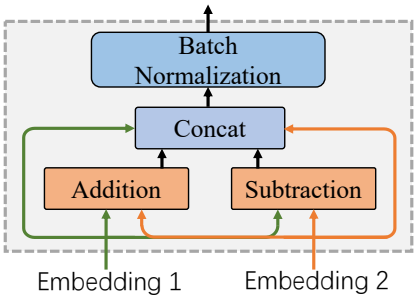



Fig. 5: Embedding interaction block

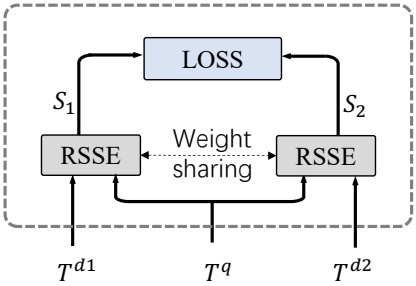



Fig. 6: Siamese architecture of RSSE

$G^q$ for the point $G_i^d$. Similarly, we can obtain the embedding of the matching point in $G^d$ for $G_j^q$, denoted as $M_j^q$.

To streamline the calculation, we perform a matrix multiplication of $G^q$ with the transpose of matrix $G^d$ to create an attention matrix $A \in \mathbb{R}^{n\times m}$. The $j$-th row of this matrix represents the degree of association between $G_j^q$ and the elements in $G^d$, while the $i$-th column represents the association between $G_i^d$ and the elements in $G^q$. Considering zero-padding for uniform trajectory lengths within a batch, a masking step zeroes out padding impacts in $A$, maintaining attention on actual data points. By applying softmax row-wise (for $G_j^q$) and column-wise (for $G_i^d$), we normalize $A$ to obtain weights. These weights help sum up embeddings of points in $G^d$ and $G^q$ to derive matching embeddings $M_j^q$ and $M_i^d$. The whole process can be formulated as follows:

$$A = \text{mask}(G^q \cdot (G^d)^\top) \tag{8}$$

$$W^{rj} = \text{softmax}(A_j, \text{row}), \quad W^{ci} = \text{softmax}(A_i, \text{col}) \tag{9}$$

$$M_j^q = \sum_{k=1}^{m} W_k^{rj} \cdot G_k^d, \qquad M_i^d = \sum_{k=1}^{n} W_k^{ci} \cdot G_k^q \tag{10}$$

where $A \in \mathbb{R}^{n\times m}$ is the attention matrix after the mask operation, $W^{ri} \in \mathbb{R}^{1\times m}$ and $W^{cj} \in \mathbb{R}^{1\times n}$ denote the weight coefficient matrices obtained through softmax normalization applied to $j$-th row and $i$-th column of $A$, respectively. $M^q \in \mathbb{R}^{n\times d}$ and $M^d \in \mathbb{R}^{m\times d}$ represent the corresponding matching embeddings in the analysis.

### C. Similarity Feature Extraction Module

Leveraging the point-level matching module, we have derived the matching embeddings $M^q$ and $M^d$ respectively. As shown in Figure 4, the distance to these nearest neighbor points(matching embedding), facilitates the differentiation between representative similar subtrajectories and irrelevant points. Therefore, to capture the difference between the embeddings and their corresponding matches, and to unearth potential similarities, we introduce an embedding interaction block, illustrated in Figure 5.

This block utilizes element-wise addition and subtraction to distill insights from the embeddings. Specifically, element-wise subtraction elucidates the variances across dimensions between two embeddings, aiding the model in identifying salient features through a differential map. Conversely, element-wise addition amalgamates attributes from two distinct sources, enhancing the exploration of latent similarities. Drawing inspiration from residual neural networks [19], we amalgamate the outcomes of these operations with the original embeddings to formulate a comprehensive feature representation within the embedding interaction context. Given the potential for these operations to produce outsized values, which could impede network convergence and impact performance, we incorporate Batch Normalization (BN) to normalize the embeddings, thereby facilitating faster convergence and bolstering the model's generalization capacity. In the case of $G^q$, the interaction process is as follows:

$$\begin{aligned} F^q &= BN(\text{concat}(G^q, M^q, G^q + M^q, G^q - M^q)) \\ &= Interaction(G^q, M^q) \end{aligned} \tag{11}$$

where $F^q \in \mathbb{R}^{n\times(4d)}$ represents the embedding features extracted from the embedding interaction block that are related to $G^q$. The extraction process in the embedding interaction block is denoted as $Interaction(\cdot,\cdot)$. We use the same process to obtain $F^d \in \mathbb{R}^{m\times(4d)}$.

The point-level difference features and potential similarity information contained in $F^q$ and $F^d$ are crucial, especially the sequence relationships within them. Therefore, we employ a Bi-GRU with new parameters to extract the sequential information within them as Equation 12, where $\hat{F}^q \in \mathbb{R}^{n\times d}$ and $\hat{F}^d \in \mathbb{R}^{m\times d}$ represent the features extracted based on $G^q$ and $G^d$, respectively.

$$\hat{F}^q = \text{Bi-GRU}(F^q), \hat{F}^d = \text{Bi-GRU}(F^d) \tag{12}$$

We use max-pooling and mean-pooling to get a uniform feature representation. As our preliminary analysis suggests, embedding interactions can yield significant benefits. Therefore, we apply the embedding interaction block to the max-pooling vectors based on $F^q$ and $F^d$, as well as to the mean-pooling vectors, achieving more refined feature extraction. In processing the max-pooling and mean-pooling, we capture and refine element-level differences using element-wise substraction, optimizing them through Batch Normalization. Finally, we combine them with the output of embedding interactions, forming a comprehensive joint representation for



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

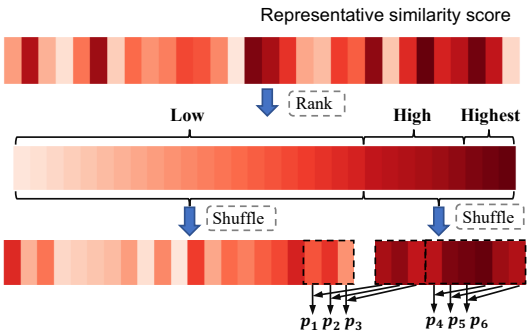



Fig. 7: Example of pair construction by sampling

the estimation of representative similarity scores,the whole process can be formulated as below:

$$F^{\hat{q}}_{max} = \text{Max-Pool}(\hat{F^q}), F^{\hat{q^{\wedge}}}_{mean} = \text{Mean-Pool}(\hat{F^q}) \quad (13)$$

$$F^{\hat{d}}_{max} = \text{Max-Pool}(\hat{F^d}), F^{\hat{d^{\wedge}}}_{mean} = \text{Mean-Pool}(\hat{F^d}) \quad (14)$$

$$H = \text{concat}\{Interaction(F^{\hat{q}}_{max}, F^{\hat{d}}_{max}), \quad (15)$$
$$Interaction(F^{\hat{q^{\wedge}}}_{mean}, F^{\hat{d^{\wedge}}}_{mean}),$$
$$BN(\text{concat}[(F^{\hat{q}}_{max} - F^{\hat{q^{\wedge}}}_{mean}), (F^{\hat{d}}_{max} - F^{\hat{d^{\wedge}}}_{mean})])\}$$

where $F^{\hat{q}}_{max}, F^{\hat{q^{\wedge}}}_{mean}, F^{\hat{d}}_{max}, F^{\hat{d^{\wedge}}}_{mean} \in \mathbb{R}^{1\times d}$ are the result of $\hat{F^q}$and $\hat{F^d}$ being subjected to both the max pooling operation and the mean pooling operation, resulting in their unification to the same dimension. $H \in \mathbb{R}^{1\times 10d}$ is the joint feature representation. Finally, we have encoded all features and then apply a three-layer fully connected neural network to generate the estimation. The associated process is formulated as:

$$\text{score} = w_3 \cdot \sigma\left(w_2 \cdot \sigma\left(w_1 H + b_1\right) + b_2\right) + b_3 \quad (16)$$

where $\sigma(.)$ is the sigmoid function, $w_1 \in \mathbb{R}^{2d\times 10d}$ and $b_1 \in \mathbb{R}^{(2d)}$ denote the weight matrix and bias parameters of the first fully connected neural network layer, $w_2 \in \mathbb{R}^{d\times 2d}$ and $b_2 \in \mathbb{R}^d$ indicate the affiliated parameters of second layer, $w_3 \in \mathbb{R}^{1\times d}$ and $b_3 \in \mathbb{R}^1$ represents parameters of the output layer. The output is the estimated representative similarity score.

### D. Optimization and Sampling

**Optimization.** The representative similarity score prediction problem is fundamentally a regression task, for which the Mean Squared Error (MSE) is a natural choice as the loss function, denoted as $L_{\text{regression}}$. A lower MSE indicates a smaller discrepancy between the predicted scores and the ground truth, signifying better prediction accuracy. However, in addition to accurate score prediction, the ranking of the predicted scores is crucial for filtering and candidate selection applications. To address this aspect, we introduce a ranking loss component, $L_{\text{ranking}}$, which penalizes deviations in the ranking of the predicted scores.

During the training phase, we employ a Siamese network architecture, as illustrated in Figure 6. Given a query trajectory $T^q$ and a pair of data trajectories $T^d_1$ and $T^d_2$, we create trajectory pairs by associating $T^q$ with each of $T^d_1$ and $T^d_2$. Two instances of the RSSE model, sharing identical weights, are used to predict the similarity scores $S^1$ and $S^2$ for these pairs. The ranking loss is calculated as follows:

$$L_{\text{ranking}} = \max(0, -(\hat{S}^1 - \hat{S}^2)(S^1 - S^2)) \quad (17)$$

where $\hat{S}^1$ and $\hat{S}^2$ are the actual representative similarity scores. The ranking loss is zero when the predicted order is consistent with the actual order, effectively nullifying the penalty. Conversely, any misalignment between the predicted and actual ranking incurs a penalty proportional to the degree of inconsistency.

The overall loss function for optimizing RSSE is a combination of the regression loss and the ranking loss as Equation 18. By incorporating both the regression loss and the ranking loss, we ensure that RSSE not only predicts accurate scores but also focuses on their ranking . This optimization strategy enhances RSSE's effectiveness in candidate filtering and selection applications, allowing RSSE to learn a meaningful representation of the representive similarity scores while preserving the relative ranking of the trajectories. Ultimately, this improves its performance in real-world scenarios.

$$Loss = L_{\text{regression}} + L_{\text{ranking}} \quad (18)$$

**Sampling.** Effective data sampling is essential for model optimization. In some learning-based methods, Neutraj employs a strategy of selecting the nearest $k$ trajectories as positive samples and the furthest $k$ trajectories as negative samples to form training pairs. Similarly, TMN opts for randomly selecting $2k$ trajectories, establishing similarity rankings for the first and latter $k$ sets, thus generating pairs. Despite their utility, these methods have limitations in accurately predicting the top-$k$ results.

Firstly, the absence of samples with high representative similarity scores in the combination pairs might hinder the full potential of $L_{\text{ranking}}$, leading to insufficient model discrimination for top-$k$ data, thereby impeding the efficient selection of candidate trajectory sets. Secondly, the stark contrast between trajectories that have high representative similarity scores and those with low representative similarity scores with the query trajectory makes them relatively easy to distinguish. The challenge lies in identifying those trajectories that have a higher representative similarity but still do not need to be considered as candidates. Thirdly, for trajectories with low representative similarity, their prediction accuracy and the importance of their ranking are less significant.

To overcome these limitations, we propose a novel hierarchical sampling technique, depicted in Figure 7. This method allows for efficient training with a minimal dataset, thereby optimizing the utilization of the ranking loss and significantly improving the model's capacity to discern between subtrajectories that exhibit high representative similarity scores. Assuming $t$ data trajectories are chosen for each query for model training, we begin by ranking data trajectories by their representative similarity scores, selecting the top $3/4t$, which includes those with subtrajectories most similar to the



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

query. These $3/4t$ trajectories are shuffled, with $2/4t$ randomly selected to form pairs, focusing the model's training on high-scoring samples. The remaining trajectories are also shuffled, with $t/4$ randomly chosen to pair with the residual $t/4$ from the initial selection. This enhances the model's ability to distinguish between high and low-scoring samples with limited data.

Given the variability in the number of data trajectories that contain subtrajectories similar to the query, employing a fixed value for $t$ may not be optimal. Therefore, we introduce a hyperparameter sampling rate $r$ , where $0 < r < 1$, to modulate the approach for identifying potential top-$k$ candidates as follow:

$$t = c_q * r \tag{19}$$

where $c_q$ denotes the count of data trajectories relevant to the query. This adaptable strategy ensures a more flexible and nuanced method for sample selection, enhancing the model's efficacy in predicting top-$k$ results with improved precision.

## IV. Experiments

The experiment evaluated the proposed methods from two aspects: (1) The performance of RSSE and other learning-based models in accurately predicting the top-$k$ representative similarity scores. (2) The quality of the top-$k$ representative similar subtrajectories obtained by the learning-based filter and other filtering methods.

### A. Experiment Setting

**Datasets.** All experiments were conducted on real trajectory datasets from three cities: Xi'an, Chengdu and Porto. The DiDi Chuxing GAIA Open Dataset [20] provides taxi order data for Xi'an in October 2016 and Chengdu in November 2016, with a total of 3.1 million and 5.8 million trajectories for the two cities, respectively. The Porto dataset [21] contains more than 1.7 million trajectories, describing a whole year (i.e., from July 1st, 2013 to June 30th, 2014) of the trajectories for all the 442 taxis running in the city of Porto, Portugal. Following previous works [12] [14], we have conducted preprocessing of these datasets. For the Porto dataset, trajectories from remote areas are filtered out, retaining only those densely located in the city center. In all datasets, trajectories with apparent errors are excluded. After preprocessing, the detailed information of the three datasets is presented in Table I.

TABLE I: Dataset statistics

| Dataset | Xi'an | Chengdu | Porto |
|---|---|---|---|
| Number | 85802 | 221781 | 1252547 |
| Mean length | 104 | 97 | 43 |
| Latitude | 34.21 ~ 34.28 | 30.65 ~ 30.73 | 41.11 ~ 41.2 |
| Longitude | 108.91 ~ 109 | 104.04 ~ 104.13 | -8.68 ~ -8.57 |
| Interval (s) | 3 | 3 | 15 |

Based on Definition 3, data trajectories are usually longer than query trajectories. It's not very productive to search for subtrajectories in trajectories shorter than the query. Consequently, in our experiments, we always make sure that data trajectories are longer than query trajectories. We keep the length of query trajectories between 30 and 90, and the length of data trajectories between 90 and 300. This setup allows for a comprehensive evaluation of the method's performance across varying trajectory lengths, with data trajectories being 1 to 10 times the length of the query trajectories. For the experimental dataset, we randomly chose 30,000 trajectories to serve as data trajectories and 2,000 trajectories as query trajectories.

**Metrics.** In order to evaluate the performance of the RSSE model and the learning-based filter, we employ two sets of metrics as following.

*(a) For RSSE.* We evaluate the performance of RSSE and its competitors through three distinctive metrics. NDCG@50, a staple in evaluating information retrieval and ranking systems, measures the normalized gain of the top-50 predictions against their actual rankings, highlighting the model's precision. HR-50 examines the match between the top-50 predictions and the actual top-50 scores, reflecting the model's accuracy in identifying closely similar data points. R10@50 assesses the overlap between the top-50 predictions and the actual top-10 scores, offering insights into the model's ability to discern the most relevant similarities. All metrics range from 0 to 1, with values nearing 1 indicating better performance.

*(b) For Filter.* We employ the Relative Rank (RR) metric to assess the quality of top-$k$ subtrajectories provided by different methods. Given a query trajectory, various methods generate their respective top-$k$ subtrajectories. These subtrajectories are subsequently ordered, and RR reports the normalized average ranking of the top-$k$ subtrajectories as determined by the different methods. A smaller RR value indicates better performance.

**Identification methods.** We employed ExactS [16] to calculate the representative similarity score with the query trajectory as the ground-truth from a data trajectory. This method involves an exhaustive enumeration of all possible subtrajectories, computing their similarity with the query trajectory, thereby deriving the representative similarity score. PSS [16] segments trajectories based on their points and calculates similarity with the query trajectory, avoiding full enumeration. This method is faster but yields approximate results. RLS [16], similar to PSS, uses trajectory segmentation but incorporates reinforcement learning for intelligent segmentation point determination. This method balances accuracy and computational efficiency.

**Baseline methods.** To evaluate the efficacy of the RSSE in predicting representative similarity scores, we adapted existing learning-based trajectory similarity estimation methodologies as baselines to compare with RSSE. Furthermore, to assess the performance of the learning-based filtering techniques proposed in this study, we selected two filtering methods as baselines for comparison.

*(a) For RSSE.* (1). SRN [22] is based on siamese architecture and employs recurrent neural network to process time series. Following previous works [12] [13], we implement SRN with LSTM and apply it to estimate representative similarity scores. (2). T3S [13] uses a self-attentive based network to process the trajectory of the Grid representation and LSTM to encode



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Performance of different learning-based methods under popular trajectory similarity measurement.

| Distance Function | | DTW | | | EDR | | | ERP | | | LCSS | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Dataset | Method | NDCG@50 | R10@50 | HR50 | NDCG@50 | R10@50 | HR50 | NDCG@50 | R10@50 | HR50 | NDCG@50 | R10@50 | HR50 |
| Xi'an | SRN | 0.5384 | 0.0803 | 0.0762 | 0.5387 | 0.0760 | 0.0756 | 0.5478 | 0.0855 | 0.0862 | 0.5490 | 0.1041 | 0.0904 |
| | T3S | 0.5079 | 0.0594 | 0.0580 | 0.5080 | 0.0585 | 0.0577 | 0.5078 | 0.0555 | 0.0577 | 0.5076 | 0.0655 | 0.0558 |
| | TMN | 0.6258 | 0.2332 | 0.2045 | 0.6452 | 0.3171 | 0.2463 | 0.6479 | 0.3049 | 0.2507 | 0.6470 | 0.3030 | 0.2482 |
| | RSSE | **0.7568** | **0.5585** | **0.4150** | **0.8410** | **0.7949** | **0.5744** | **0.8303** | **0.6980** | **0.5529** | **0.8542** | **0.7169** | **0.5907** |
| Chengdu | SRN | 0.5427 | 0.1316 | 0.1054 | 0.5583 | 0.1437 | 0.1179 | 0.5983 | 0.2193 | 0.1763 | 0.5725 | 0.1724 | 0.1506 |
| | T3S | 0.4984 | 0.0638 | 0.0636 | 0.4973 | 0.0600 | 0.0622 | 0.4970 | 0.0628 | 0.0627 | 0.4991 | 0.0871 | 0.0644 |
| | TMN | 0.6016 | 0.2507 | 0.1932 | 0.6141 | 0.2441 | 0.1940 | 0.6395 | 0.2974 | 0.2369 | 0.6405 | 0.2964 | 0.2411 |
| | RSSE | **0.7459** | **0.5510** | **0.4175** | **0.8057** | **0.7315** | **0.5153** | **0.8555** | **0.7343** | **0.6030** | **0.8493** | **0.7098** | **0.5919** |
| Porto | SRN | 0.4461 | 0.0362 | 0.0338 | 0.3988 | 0.0105 | 0.0096 | 0.5090 | 0.1171 | 0.1002 | 0.4128 | 0.0135 | 0.0116 |
| | T3S | 0.3844 | 0.0093 | 0.0080 | 0.3828 | 0.0070 | 0.0067 | 0.3820 | 0.0065 | 0.0066 | 0.3862 | 0.0088 | 0.0075 |
| | TMN | 0.5372 | 0.1760 | 0.1325 | 0.4691 | 0.0634 | 0.0496 | 0.5359 | 0.1710 | 0.1304 | 0.4675 | 0.0638 | 0.0537 |
| | RSSE | **0.7908** | **0.6455** | **0.4886** | **0.8344** | **0.8114** | **0.5609** | **0.8469** | **0.7671** | **0.5818** | **0.8580** | **0.8554** | **0.6023** |

a sequence of coordinates containing spatial information. (3). TMN [12] is the state-of-the-art method for trajectory similarity learning. It uses an attention mechanism to capture the connection between points in two trajectories.

*(b) For Filter.* (1). OSF: It is the current state-of-the-art filtering method in the search for similar subtrajectories in road networks. We extend this approach from road networks to euclidean space (2). Grid Filter: This method segments the spatial domain into a Grid of equal-sized cells. Data trajectories are then assigned to these cells based on their spatial paths. When a query trajectory is provided, it is mapped to the corresponding Grid, and trajectories that do not pass through these Grids are filtered. (3). LBF(Learning-Based Filter): LBF utilizes the representative similarity scores predicted by RSSE to select candidates, with the default number of candidates being five times $k$. To avoid unnecessary predictions, we initially employ Grid filter to eliminate trajectories.

**Parameter settings.** All the methods are implemented in Python 3.9. The implementation of the learning-based method is based on Pytorch 1.11. We use Adam [23] with a weight decay of 1e-4 as the optimizer in RSSE . The learning rate (lr) of RSSE and the dimension of the Bi-GRU output vectors $d$ are set to 0.001 and 64, respectively. The training process utilized an early stopping mechanism, terminating the training if there was no improvement in performance on the validation set for 10 consecutive epochs. Following previous work [14], the Grid size is set to 50m x 50m. We used DTW, ERP, EDR and LCSS distance introduced in Section II-B as similarity metrics. From the experiment results, we observe that the performance for LCSS distance is quite similar to EDR and due to space limitation, we only report LCSS distance results in Table II and III. For other experiments, we report the result for the first three metrics as default. The experiments are conducted on a server with Intel(R) Xeon(R) Silver 4216 CPU @ 2.10GHz 256.00GB RAM and one Nvidia Tesla V100 GPU. The platform ran on Ubuntu 20.04 OS.

## B. RSSE Performance

*1) Effectiveness.*

To evaluate the predictive capability of different model for unknown query trajectories, we partitioned the query trajectory dataset into training, validation, and test sets with respective ratios of 0.45, 0.05, and 0.5. For each query trajectory, Grid filter is employed to retain data trajectories with spatial

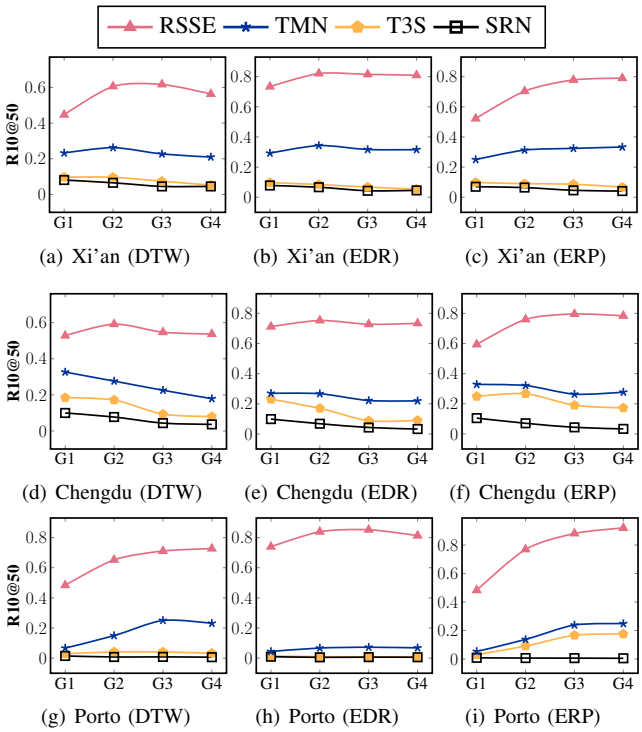



Fig. 8: The performance of queries with different length intervals

similarities, thereby reducing inefficient searches. The ExactS method is used to calculate the representative similarity scores between the filtered trajectories in the database and the query trajectory as the ground truth. This partitioning strategy aims to emulate real-world querying scenarios and comprehensively assess the model's generalization ability.

The results, detailed in Table II, demonstrate that RSSE significantly outperforms all baseline methods across all metrics on three datasets. Specifically, SRN, which encodes entire data trajectories, inadvertently embeds irrelevant points, misleading the estimation of representative similarity scores and resulting in lower accuracy. Although T3S introduces self-attention to enhance feature extraction within trajectories, the internal information of trajectories fails to contribute effectively to the estimation of representative similarity scores. While TMN's matching-based approach surpasses SRN and T3S, its coarse-grained feature extraction significantly undermines its performance compared to RSSE.



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Ablation study

| Distance Function | | DTW | | | EDR | | | ERP | | | LCSS | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Dataset | Method | NDCG@50 | R10@50 | HR50 | NDCG@50 | R10@50 | HR50 | NDCG@50 | R10@50 | HR50 | NDCG@50 | R10@50 | HR50 |
| Chengdu | RSSE-Match | 0.6099 | 0.2310 | 0.1886 | 0.6407 | 0.3033 | 0.2310 | 0.6747 | 0.3385 | 0.2766 | 0.6935 | 0.4007 | 0.3122 |
| | RSSE-Inter | 0.6600 | 0.3660 | 0.2729 | 0.6826 | 0.4324 | 0.3030 | 0.7081 | 0.4254 | 0.3347 | 0.7196 | 0.4687 | 0.3607 |
| | RSSE-Mse | 0.6968 | 0.4446 | 0.3235 | 0.7845 | 0.6833 | 0.4737 | 0.8157 | 0.6543 | 0.5248 | 0.7984 | 0.6137 | 0.4994 |
| | RSSE | **0.7459** | **0.5510** | **0.4175** | **0.8057** | **0.7315** | **0.5153** | **0.8555** | **0.7343** | **0.6030** | **0.8493** | **0.7098** | **0.5919** |

RSSE with its point-level matching module, adeptly locates the representative similar subtrajectory, utilizing an attention mechanism that assigns low weight to irrelevant points while preserving information about similar subtrajectories. The similarity extraction module captures sequence and similarity relations, contributing to RSSE's superior performance in subtrajectory similarity approximation. Specifically, RSSE outperforms the second-best method, TMN, by approximately doubling the average R10@50 score, indicating that RSSE can effectively identify trajectories with the highest true representative similarity scores. In comparison to TMN, RSSE achieves an average improvement of 0.23 in NDCG@50 and an average improvement of 0.37 in HR50. These results suggest that although the top-$k$ results predicted by RSSE may not precisely match the actual top-$k$, they still maintain a considerably high quality. RSSE's advantage is most prominent in the Porto dataset, which can be attributed to the low sampling frequency and broader spatial coverage of trajectories in this dataset. These factors increase the difficulty of predicting representative similarity scores for the models. While the performance of other methods exhibits a noticeable decline under these conditions, RSSE consistently delivers excellent results, demonstrating its generalization ability and robustness under diverse data characteristics.

To demonstrate the effect of query trajectory length, we segmented the query trajectories into four groups, denoted as G1 to G4, based on their lengths as follows: G1 = [30, 45), G2 = [45, 60), G3 = [60, 75), and G4 = [75, 90). For each group, we present the average result. As shown in Figure 8, RSSE consistently outperforms baseline methods in every group. We observed a slight decrease in accuracy for shorter queries (G1), likely due to the higher number of similar subtrajectories in the database for shorter queries, making them less distinguishable. Despite this, the quality of these subtrajectories remains high. For longer queries, the distinction between very similar subtrajectories becomes clearer, further enhancing our method's accuracy.

In summary, RSSE provides high-quality subtrajectory similarity estimation, ensuring the selection of quality candidates by the learning-based method. This enables the learning-based framework to output high-quality results, even with a minimal number of candidates.

*2) Ablation study.*

RSSE estimate representative similar scores by employing the point-level matching module and extracting fine-grained similarity features by the embedding interactions block. Furthermore, we have specifically designed loss function and sampling techniques. To demonstrate the effectiveness of these designs, ablation experiments have been conducted. A summary of the configurations applied in these methods is presented as follows:

- **RSSE-Match:** This method eliminates the point-level matching module found in RSSE, directly utilizing trajectory embeddings to extract salient features.
- **RSSE-Inter:** This method omits the embedding interactions block in the similarity extraction module of RSSE.
- **RSSE-Mse:** In this method, random trajectory pairs are chosen for training, with the Mean Squared Error (MSE) employed as the loss function.

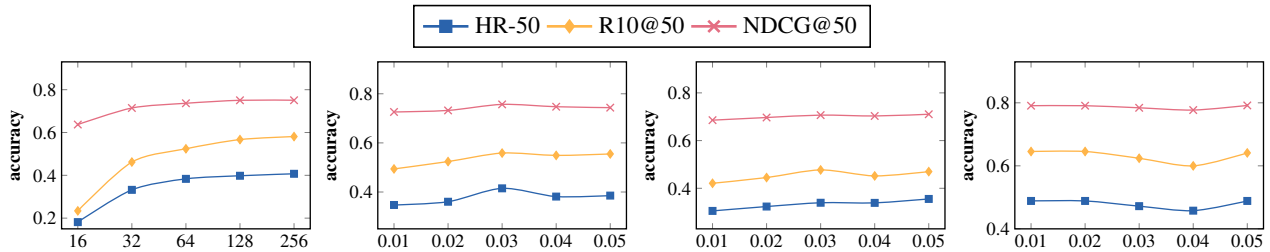
The experimental results, as shown in Table III, reveal that the prediction accuracy of RSSE-Match significantly decreases, indicating that point-level matching is crucial for predicting representative similarity scores. This matching process mitigates the impact of irrelevant points and generates matching embeddings, providing additional similarity information that lays the groundwork for subsequent feature extraction. The results of RSSE-Inter demonstrate that the embedding interactions block also plays a vital role by extracting fine-grained similarity features to deeply investigate the relationships between trajectories. The synergy between these two components enables the model to effectively estimate representative similarity scores. Furthermore, the experiments demonstrate that our designed loss function and sampling strategy enhance the model's performance and training efficiency, focusing the model on estimating high-quality candidate objects. By prioritizing the most informative samples during training and optimizing the model to minimize both the regression and ranking losses, our approach ensures that RSSE can accurately identify the most representative similar subtrajectories while maintaining a high level of computational efficiency.

*3) Parameter Analysis.*

We conduct sensitivity experiments on two crucial parameters: the hidden layer size $d$ and the sampling rate $r$. The dimension $d$ affects the model's complexity, while the value of $r$ results in training data of varying sizes and different sets of ranking pairs. These parameter sensitivity experiments were performed across multiple datasets under DTW measurement, with all other parameters set to default values. As shown in Figure 9(a), when $d$ is too small, the model underperforms due to its inability to learn effectively. As $d$ increases, the model's accuracy improves until it stabilizes. However, larger values of $d$ also imply increased model complexity. Considering this, we chose 64 as the default value for $d$. With similar observations across the other two datasets, we present results solely for the Xi'an dataset. As shown in Figures 9(b),9(c),9(d), the results indicate that variations in $r$ significantly impact the model's performance, with the most notable fluctuations observed in

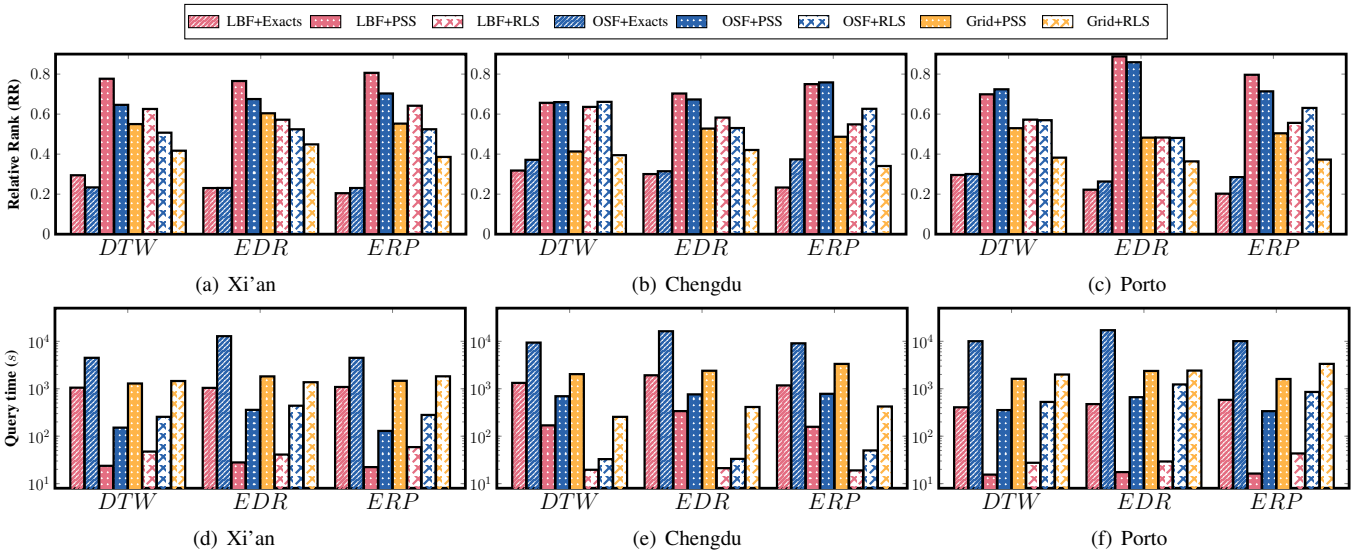


Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.




(a) Dimention $d$ (Xi'an) (b) Sampling rate $r$ (Xi'an) (c) Sampling rate $r$ (Chengdu) (d) Sampling rate $r$ (Porto)

Fig. 9: The performance of RSSE with different dimension $d$ and sampling rate $r$




(a) Xi'an (b) Chengdu (c) Porto

(d) Xi'an (e) Chengdu (f) Porto

Fig. 10: Performance of different filtering methods

the Xi'an dataset. The performance of the model initially increases and then decreases with the increase in $r$. This trend is due to the lack of sufficient training data when $r$ is small, leading to poor performance. However, as $r$ increases, the training trajectory pairs include more distinguishable data trajectories. This not only fails to enhance the model's ability to differentiate high-quality trajectories but also wastes resources on predicting low-quality trajectories, resulting in a decline in overall model quality. For the Xi'an and Chengdu datasets, the optimal values of $r$ are 0.03 and 0.02, respectively, while for the Porto dataset, the optimal value of $r$ is 0.01. This suggests that the sampling method designed for the RSSE model requires only a small amount of training data to achieve good performance, significantly enhancing training efficiency.

### C. Filtering performance

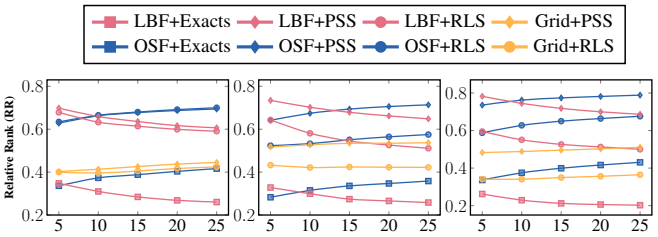
#### 1) Effectiveness and Efficiency

Figure 10 demonstrates the quality of top-$k$ results achieved by three filtering methods during the validation phase when combined with different approaches to identifying representative similar subtrajectories. When employing ExactS for validation, the LBF method consistently outperforms other methods in terms of RR scores across most datasets, with its effectiveness most noticeable under the ERP metric. This enhanced performance is attributable to OSF's inadvertent exclusion of trajectories with high representative similarity scores in an effort to reduce the candidate size. The RR score comparison suggests that, relative to OSF, LBF provides a superior quality of candidates. Moreover, LBF exhibits a marked advantage in query processing time over OSF and the Grid filter across all datasets, a distinction that is most apparent in the Porto dataset. In Porto, the lower sampling frequency and wider trajectory coverage result in OSF including more low-quality candidates, thereby significantly extending the query time compared to LBF. On average, OSF exceeds the query time of LBF by 11,602 seconds across various similarity metrics.

When employing the PSS and RLS validation methods, LBF demonstrates comparable quality to OSF in the Chengdu and Porto datasets, although with a slight underperformance in the Xi'an dataset. It reflects that during the validation phase, employing heuristic methods, a higher quality of candidates does not necessarily translate into superior top-$k$ result quality.The Grid filter retains the majority of candidates, thus minimizing the risk of inadvertently filtering out high-quality candidates, leading to the highest accuracy among the methods. However, this comes at the cost of significant computational overhead. The query time for the Grid filter is substantially longer than that for both LBF and OSF, averaging about two orders of magnitude higher than LBF and nearly an order of magnitude higher than OSF.

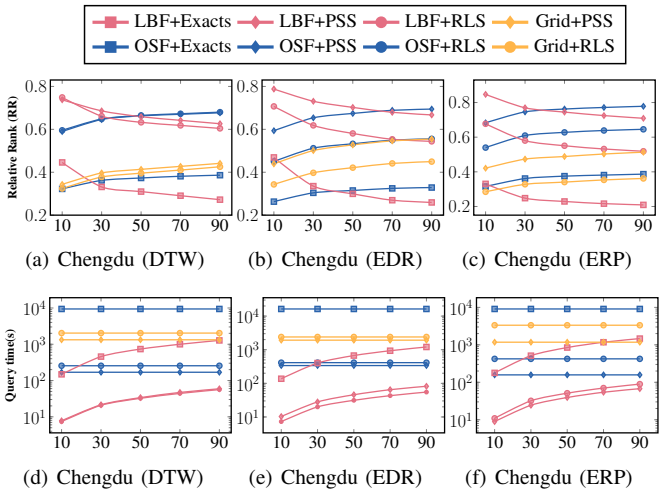


Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.




(a) Chengdu (DTW) (b) Chengdu (EDR) (c) Chengdu (ERP)

Fig. 11: Different $k$




(a) Chengdu (DTW) (b) Chengdu (EDR) (c) Chengdu (ERP)

(d) Chengdu (DTW) (e) Chengdu (EDR) (f) Chengdu (ERP)

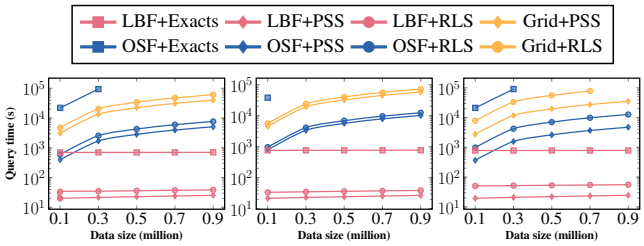
Fig. 12: Different candidate size

Thanks to RSSE's remarkable ability to estimate representative similarity scores, LBF consistently yields high-quality candidates across different trajectory similarity metrics, demonstrating its versatility and adaptability. Additionally, LBF excels in query time while still maintaining superior filtering quality compared to OSF. Contrary to the Grid filter, which prioritizes accuracy potentially at the expense of efficiency, LBF achieves a significant efficiency boost with an acceptable trade-off in accuracy.

*2) Parameter Analysis*

In this section, we analyze the performance of the compared methods under different parameter values, focusing on two parameters: The value of $k$ and the size of the candidate set. Due to the similarity of results across different datasets, we report only the results for the Chengdu dataset.

Figure 11 illustrates the performance of all compared methods at different values of $k$. As $k$ increases, the RR of LBF methods decreases. This can be attributed to the fact that LBF selects 5 times $k$ candidates, and when $k$ is small, the candidate set is also small. Although learning-based methods can choose high-quality candidates, in scenarios with many trajectories having high representative similarity scores, the top-$k$ objects often have minimal differences and are challenging to distinguish. As $k$ increases, LBF's candidate set also grows, including more high-quality candidates, thereby improving the results. Conversely, as $k$ increases, the risk of OSF erroneously deleting high-quality candidates also rises, leading to a decline in its performance. As previously mentioned, directly selecting




(a) Chengdu (DTW) (b) Chengdu (EDR) (c) Chengdu (ERP)

Fig. 13: Scalability

$5k$ candidates may not be the optimal choice.

Another important parameter is the candidate size. A smaller number of candidates may lead to the erroneous exclusion of high-quality candidates, while increasing the number of candidates will lead to an increase in computation cost. We evaluate the effect of different candidate size and the results are shown in Figure 12. The quality of LBF is notably lower when the candidate size is set to 10 compared to OSF and Grid methods. Increasing the candidate size enhances the quality of LBF results across all validation methods, improving the overall accuracy (Relative Rank). However, as the number of candidates continues to increase, the improvement in result quality gradually diminishes. For instance, in Figure 12 (a), the reduction in RR score resulting from increasing the candidate size from 30 to 50 is much less significant than that from increasing the candidate size from 10 to 30. Therefore, we recommend gradually increasing the candidate size and stopping when the incremental improvement becomes insignificant (as depicted by the flattened RR score curve in Figure 12), striking a balance between result quality and computation efficiency.

*3) Scalability.*

As previously analyzed, performing TRSSQ in trajectory databases containing extensive trajectory datasets is computationally challenging. Considering this conclusion is consistent across all datasets, Figure 13 presents results exclusively for the Chengdu dataset to demonstrate the scalability of the filtering methods. In instances where query times exceed $10^5$ seconds, the results are considered to be overflows. As the dataset size grows, the number of candidates increases, resulting in a linear relationship between query time and dataset size for both OSF and Grid filter. Consequently, these methods become less feasible for large-scale datasets as the query time lengthens with increasing dataset size. In contrast, LBF selects a small number of high-quality candidates by estimating representative similarity scores, demonstrating superior scalability even in scenarios of exponential data growth. LBF's query time increases at a significantly slower rate than OSF and Grid filter, making it viable even in computationally intensive validation scenarios. By achieving an excellent balance between quality and precision, LBF's superior performance in scalability positions it as a promising solution for addressing the TRSSQ problem in large-scale data contexts.



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

## V. Related Work

**Trajectory Similarity Measurements.** Trajectory similarity measurement has a long history of research and development, with various measurement methods proposed for diverse application scenarios [24]–[27] and data characteristics. A classical approach to measure the similarity between two trajectories is to assess the degree of point matching between them. DTW [6] aligns two trajectories by finding the minimum cumulative distance between corresponding points in the trajectories. it allows for one point to correspond to multiple points, which enables it to handle trajectories with different lengths, sampling rates, or some degree of temporal distortion. Edit Distance on Real sequence (EDR) [7] aligns two trajectories by finding the minimum number of edit operations (such as insertions, deletions, and substitutions) required to transform one trajectory into the other. Edit Distance with Real Penalty (ERP) [8] is a variation of EDR that assigns different costs to each edit operation, based on the magnitude of the difference between the values being compared. The cost function can be adjusted to account for different penalties for each operation, providing flexibility in comparing trajectories. Longest Common Subsequence (LCSS) [28] computes the similarity between two trajectories by aligning them and finding the longest common subtrajectory, it allows for some spatial and temporal deviations between the trajectories. Fréchet [29] takes into account the overall shape and the order of the points along the curves.

**Learning-based Trajectory Similarity Estimation.** These similarity metrics typically require quadratic time complexity. Some learning-based methods have been proposed for fast approximate similarity calculation. They approximate trajectory similarity by measuring the distance between the embedding vectors, greatly accelerating similarity calculation. SRN [22] adopt siamese framework and employed recurrent neural network to obtain trajectory embedding. t2vec [11] implement a sequence-to-sequence architecture and takes the final hidden vector of the encoder to represent a trajectory. Neutraj [14] design a Spatial attention memory unit that memorizes the information from previously processed trajectories with the attention mechanism. T3S [13] utilizes the self-attention mechanism to further capture internal information within the trajectories. TMN [12] propose a matching mechanism that associates points across trajectory pairs, which exploits the trajectory matching information for representation learning. [30] [31] consider the trajectory similarity measure in special cases. The aforementioned methods are designed for estimating similarity between entire trajectories and cannot be used for estimating subtrajectory similarity.

**Similar Subtrajectory Searching problem.** Although similar trajectory search has been extensively studied, research on similar subtrajectory search remains relatively limited. Koide et al. [17] address the similar subtrajectory search problem in road networks. They proposed the OSF method for weighted edit distance (WED), which identifies a set of subsequence elements in the query to establish a lower bound of similarity. [16] and [32] concentrate on locating the most similar subtrajectory to a query trajectory within a given data trajectory. ExactS [16] exhaustively enumerates all subtrajectories in the data trajectory and incrementally computes their similarity to the query trajectory, resulting in high time complexity of $O(nm^2)$. Heuristic algorithms like POS PSS and RLS [16] partition the data trajectory into subtrajectories using different methods and return the most similar one to the query trajectory with complexity of $O(nm)$. CMA [32] is a more efficient method for finding the most similar subtrajectory within a data trajectory. Deng et al. [33] employ grid sequences for trajectory mapping and subtrajectory similarity study, inspired by the reading comprehension problem [34]. However, this method's use of grids as basic units instead of latitude and longitude points restricts its applicability to general similarity metrics and thus it is not included in our comparison.

**Top-$k$ Trajectory Similarity Search.** Several approaches have been proposed to solve the top-$k$ trajectory similarity search problem by constructing specialized indexes. For instance, UTgrid [35] is an effective indexing method designed for similarity search on uncertain trajectories, while RP-Trie Index [36] is tailored for organizing trajectory data to facilitate local search operations. Garden [37] employs a sophisticated distributed dynamic spatial index for continuous top-$k$ trajectory similarity search. Additionally, learning-based methods have been applied to top-$k$ trajectory similarity search. Aries [38] utilizes metric-based representation learning for fast top-$k$ trajectory similarity search, which integrates multiple metrics to cater to diverse scenarios. However, all these studies are based on trajectories and are not applicable to subtrajectories.

## VI. Conclusion

In this paper, we propose the Top-$k$ Representative Similar Subtrajectory Query problem and develop a learning-based framework that efficiently addresses this problem while providing a manageable balance between efficiency and effectiveness. This framework relies on the predictive scores of a well-designed Representative Similarity Score Estimation model to filter trajectories and select high-quality candidates. Extensive experiments conducted on real-world datasets validate the efficacy of RSSE in estimating representative similarity scores and demonstrate the potential applicability of our proposed method in large-scale data environments.

## VII. acknowledgment

Shiyu Yang is supported by National Key R&D Program of China (2022YFB3103701) and Guangzhou Basic and Applied Basic Research Foundation (202201020131). Peng Cheng is supported by the National Natural Science Foundation of China under Grant No. 62102149. Jianye Yang's work is supported by Guangdong Basic and Applied Basic Research Foundation (2023A1515011655), Science and Technology Program of Guangzhou Foundation (2023A03J0115) and Major Key Project of PCL (PCL2022A03). Xuemin Lin is supported by NSFC U2241211, NSFC U20B2046 and 23H020101910.



Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] M. Li, P. Tong, M. Li, Z. Jin, J. Huang, and X.-S. Hua, "Traffic flow prediction with vehicle trajectories," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 294–302, May 2021.

[2] B. Hui, D. Yan, H. Chen, and W.-S. Ku, "Trajnet: A trajectory-based deep learning model for traffic prediction," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery amp; Data Mining*, KDD '21, (New York, NY, USA), p. 716–724, Association for Computing Machinery, 2021.

[3] Y. Wang, Y. Zheng, and Y. Xue, "Travel time estimation of a path using sparse trajectories," in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 25–34, 2014.

[4] H. Yuan, G. Li, Z. Bao, and L. Feng, "Effective travel time estimation: When historical trajectories over road networks matter," in *Proceedings of the 2020 acm sigmod international conference on management of data*, pp. 2135–2149, 2020.

[5] L. Sha, P. Lucey, Y. Yue, P. Carr, C. Rohlf, and I. Matthews, "Chalkboarding: A new spatiotemporal query paradigm for sports play retrieval," in *Proceedings of the 21st International Conference on Intelligent User Interfaces*, IUI '16, (New York, NY, USA), p. 336–347, Association for Computing Machinery, 2016.

[6] B.-K. Yi, H. Jagadish, and C. Faloutsos, "Efficient retrieval of similar time sequences under time warping," in *Proceedings 14th International Conference on Data Engineering*, pp. 201–208, 1998.

[7] L. Chen, M. T. Özsu, and V. Oria, "Robust and fast similarity search for moving object trajectories," (New York, NY, USA), Association for Computing Machinery, 2005.

[8] L. Chen and R. Ng, *On The Marriage of Lp-norms and Edit Distance*, p. 792–803. Elsevier, 2004.

[9] L. Chen, M. T. Özsu, and V. Oria, "Robust and fast similarity search for moving object trajectories," in *Proceedings of the 2005 ACM SIGMOD international conference on Management of data*, pp. 491–502, 2005.

[10] D. Xie, F. Li, and J. M. Phillips, "Distributed trajectory similarity search," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1478–1489, 2017.

[11] X. Li, K. Zhao, G. Cong, C. S. Jensen, and W. Wei, "Deep representation learning for trajectory similarity computation," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, IEEE, 2018.

[12] P. Yang, H. Wang, D. Lian, Y. Zhang, L. Qin, and W. Zhang, "Tmn: Trajectory matching networks for predicting similarity," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, pp. 1700–1713, 2022.

[13] P. Yang, H. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "T3s: Effective representation learning for trajectory similarity computation," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, IEEE, 2021.

[14] D. Yao, G. Cong, C. Zhang, and J. Bi, "Computing trajectory similarity in linear time: A generic seed-guided neural metric learning approach," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, IEEE, 2019.

[15] R. Waury, J. Hu, B. Yang, and C. S. Jensen, "Assessing the accuracy benefits of on-the-fly trajectory selection in fine-grained travel-time estimation," in *2017 18th IEEE International Conference on Mobile Data Management (MDM)*, pp. 240–245, 2017.

[16] Z. Wang, C. Long, G. Cong, and Y. Liu, "Efficient and effective similar subtrajectory search with deep reinforcement learning," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, p. 2312–2325, 2020.

[17] S. Koide, C. Xiao, and Y. Ishikawa, "Fast subtrajectory similarity search in road networks under weighted edit distance constraints," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, p. 2188–2201, 2020.

[18] H. Su, S. Liu, B. Zheng, X. Zhou, and K. Zheng, "A survey of trajectory distance measures and performance evaluation," *The VLDB Journal*, vol. 29, pp. 3–32, 2019.

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[20] "Didi chuxing gaia open dataset." online, 2016. https://outreach.didichuxing.com/research/opendata.

[21] "Porto dataset." online, 2016. https://www.kaggle.com/c/pkdd-15-predict-taxi-servicetrajectory-i/data.

[22] W. Pei, D. M. J. Tax, and L. van der Maaten, "Modeling time series similarity with siamese recurrent networks," *ArXiv*, vol. abs/1603.04713, 2016.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014.

[24] K. Buchin, M. Buchin, D. Duran, B. T. Fasy, R. Jacobs, V. Sacristan, R. I. Silveira, F. Staals, and C. Wenk, "Clustering trajectories for map construction," in *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, SIGSPATIAL '17, (New York, NY, USA), Association for Computing Machinery, 2017.

[25] S. Wang, Z. Bao, J. S. Culpepper, T. Sellis, and X. Qin, "Fast large-scale trajectory clustering," *Proc. VLDB Endow.*, vol. 13, p. 29–42, sep 2019.

[26] Y. Wang, K. Qin, Y. Chen, and P. Zhao, "Detecting anomalous trajectories and behavior patterns using hierarchical clustering from taxi gps data," *ISPRS International Journal of Geo-Information*, vol. 7, no. 1, 2018.

[27] H. Wu, W. Sun, and B. Zheng, "A fast trajectory outlier detection approach via driving behavior modeling," in *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, CIKM '17, (New York, NY, USA), p. 837–846, Association for Computing Machinery, 2017.

[28] M. Vlachos, G. Kollios, and D. Gunopulos, "Discovering similar multidimensional trajectories," in *Proceedings 18th International Conference on Data Engineering*, pp. 673–684, 2002.

[29] T. Eiter and H. Mannila, "Computing discrete fréchet distance ," 1994.

[30] D. Yao, H. Hu, L. Du, G. Cong, S. Han, and J. Bi, "Trajgat: A graph-based long-term dependency modeling approach for trajectory similarity computation," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, (New York, NY, USA), p. 2275–2285, Association for Computing Machinery, 2022.

[31] Z. Fang, Y. Du, X. Zhu, D. Hu, L. Chen, Y. Gao, and C. S. Jensen, "Spatio-temporal trajectory similarity learning in road networks," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, (New York, NY, USA), p. 347–356, Association for Computing Machinery, 2022.

[32] J. Jin, P. Cheng, L. Chen, X. Lin, and W. Zhang, "Efficient non-learning similar subtrajectory search," *Proc. VLDB Endow.*, vol. 16, p. 3111–3123, jul 2023.

[33] L. Deng, H. Sun, R. Sun, Y. Zhao, and H. Su, "Efficient and effective similar subtrajectory search: A spatial-aware comprehension approach," *ACM Trans. Intell. Syst. Technol.*, vol. 13, apr 2022.

[34] A. W. Yu, D. Dohan, M.-T. Luong, R. Zhao, K. Chen, M. Norouzi, and Q. V. Le, "Qanet: Combining local convolution with global self-attention for reading comprehension," 2018.

[35] C. Ma, H. Lu, L. Shou, and G. Chen, "Ksq: Top-k similarity query on uncertain trajectories," *IEEE Transactions on Knowledge and Data Engineering*, vol. 25, no. 9, pp. 2049–2062, 2013.

[36] B. Zheng, L. Weng, X. Zhao, K. Zeng, X. Zhou, and C. S. Jensen, "Repose: Distributed top-k trajectory similarity search with local reference point tries," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pp. 708–719, 2021.

[37] Z. Pan, P. Chao, J. Fang, W. Chen, J. Xu, and L. Zhao, "Garden: a real-time processing framework for continuous top-k trajectory similarity search," *Knowl. Inf. Syst.*, vol. 65, p. 3777–3805, may 2023.

[38] M. Herlihy, "A methodology for implementing highly concurrent data objects," *ACM Trans. Program. Lang. Syst.*, vol. 15, pp. 745–770, November 1993.




Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 23,2026 at 07:33:16 UTC from IEEE Xplore. Restrictions apply.